

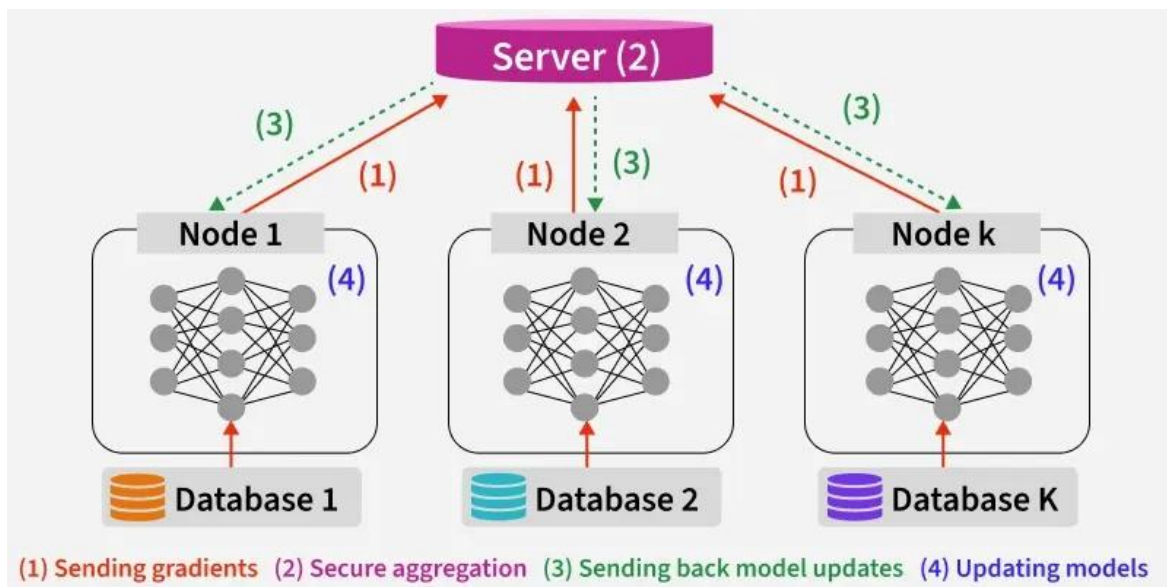
Subject: Scalable ML & BDA

Module 2: Distributed Machine Learning

Prof. Shweta Jaiswar

Distributed Deep Learning:

- Distributed Deep Learning is the practice of training huge deep neural networks by spreading the workload across multiple GPUs, TPUs, or even entire clusters.
- It's important as single devices can't handle today's massive models and datasets alone.
- Training techniques help speed up training, scale to billions of parameters and make state of the art AI possible from large language models to massive vision transformers.

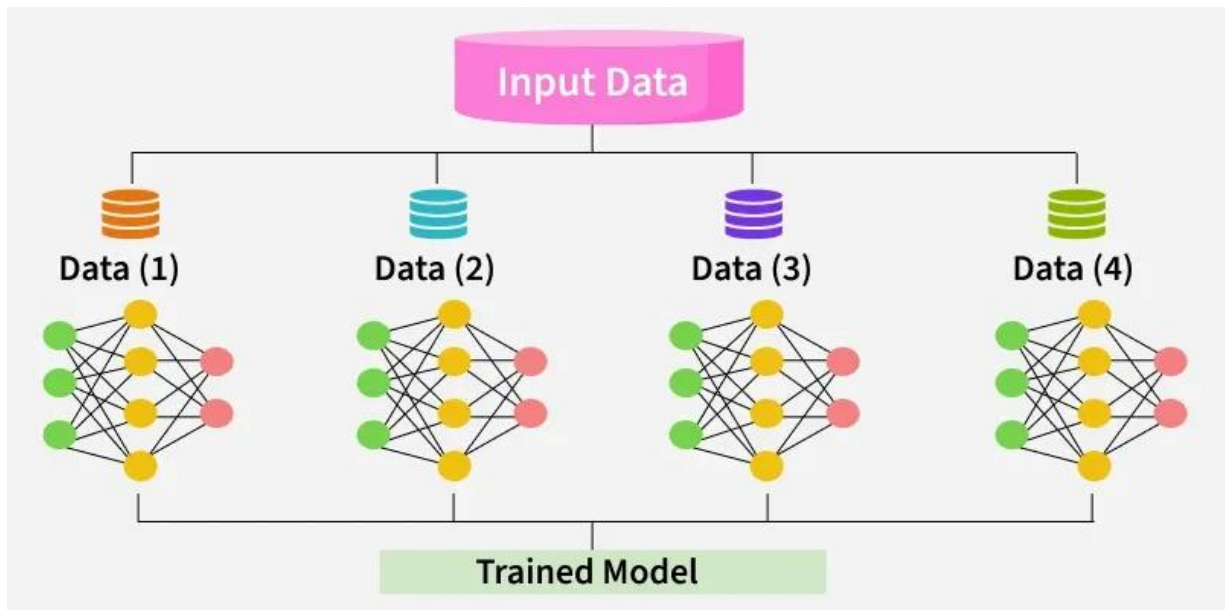


Distributed Deep Learning is the practice of dividing the training workload across multiple computational resources such as GPUs, TPUs or clusters of machines. The goal is to:

- Speed up training time often from weeks to days or even hours.
- Scale up to larger models and datasets that wouldn't fit on a single device.
- Improve resource utilization making large scale AI research practical.

Training Methods for Large-Scale Models:

1. Data Parallelism



- Data Parallelism works by replicating the entire model across multiple devices like GPUs or TPUs and then splitting the input data into separate mini batches for each device.
- Each device independently computes the forward and backward passes on its portion of the data producing gradients for its replica.
- These gradients are then communicated across devices and averaged so that every replica stays synchronized with the same updated parameters.
- This approach is especially practical when your model comfortably fits into the memory of a single device, but you need to process more data in parallel to speed up training.
- Common frameworks like PyTorch's `torch.distributed` or TensorFlow's `MirroredStrategy` make this easier to implement.

What is `torch.distributed` in PyTorch?

- Distributed computing involves spreading the workload across multiple computational units, such as GPUs or nodes, to accelerate processing and improve model performance. PyTorch's `torch.distributed` package provides the necessary tools and APIs to facilitate distributed training. This package supports various parallelism strategies, including data parallelism, model parallelism, and hybrid approaches.

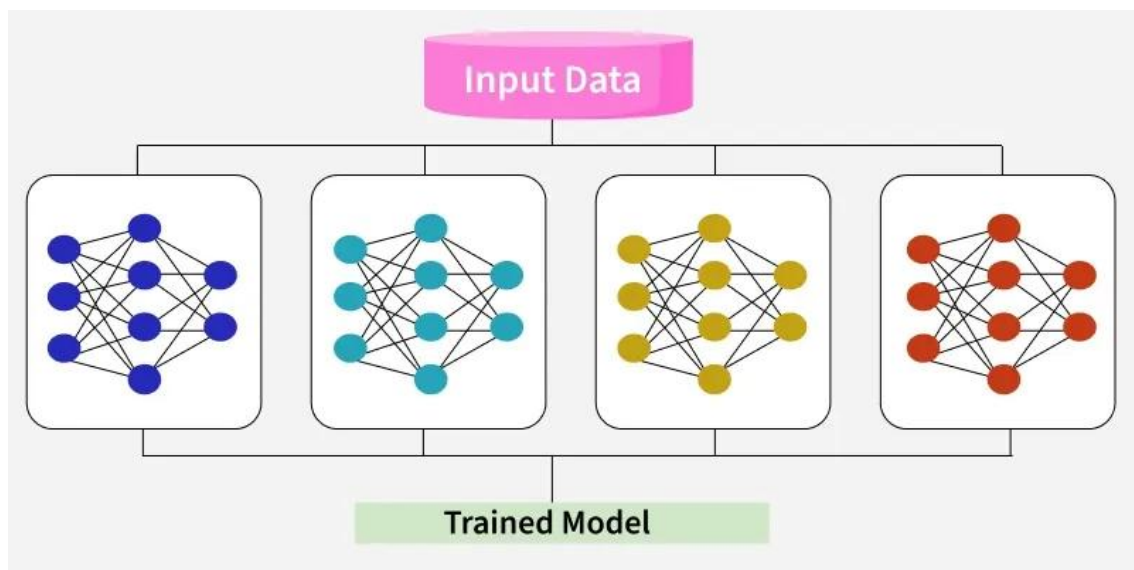
- `torch.distributed` is a package within PyTorch designed to support distributed training. Distributed training involves splitting the training process across multiple GPUs, machines, or even clusters to accelerate the training of deep learning models. By leveraging this package, users can scale their models and training processes seamlessly.

Key Concepts and Components of `torch.distributed` in Pytorch:

1. Process Groups
2. Communication Backends
3. Collective Communication
4. Distributed Data-Parallel (DDP)
5. Initialization Methods
6. Distributed Optimizers

TensorFlow's MirroredStrategy: `tf.distribute.MirroredStrategy` is TensorFlow's primary API for performing synchronous distributed training on multiple GPUs across a single machine. It replicates the entire model architecture and variables across all available processors (replicas), splits each data mini batch evenly among them, and leverages efficient all-reduce communication algorithms to aggregate gradients before applying synchronized weight updates.

2. Model Parallelism



- Model Parallelism tackles a different challenge extremely large models that simply don't fit on a single device's memory.
- Instead of replicating the model you divide it into separate parts and place each part on a different device. During training, intermediate results must be communicated between devices so the entire forward and backward passes can be completed end to end.
- This setup requires careful planning to partition the model efficiently and minimize the communication overhead.
- It's often used in training massive transformer-based language models like GPT, BERT or other large language models with billions of parameters.

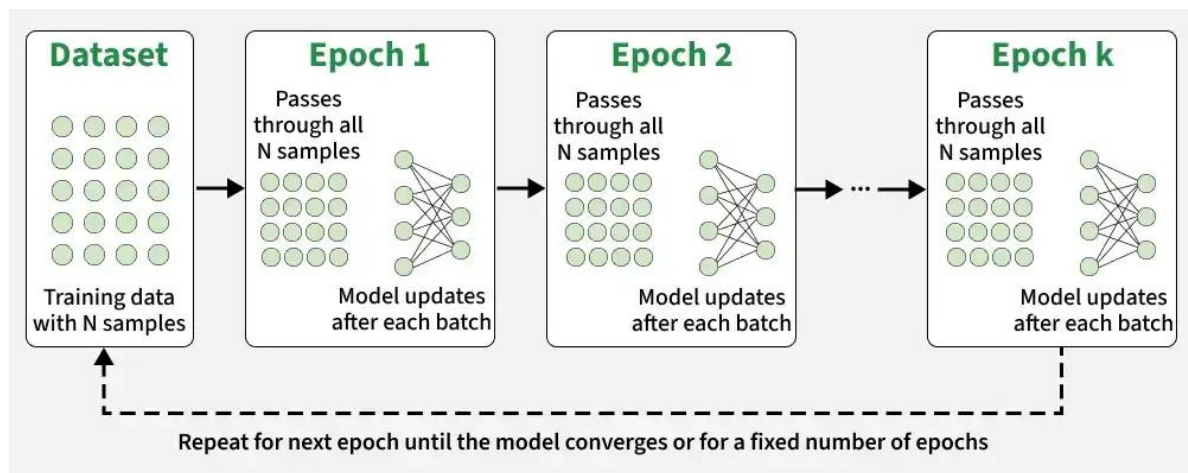
Parallel Machine Learning: Parallel Machine Learning means performing machine learning computations simultaneously using multiple processing units to reduce training time and handle large datasets.

Distributed Machine Learning: Distributed Machine Learning means training a machine learning model using multiple connected computers or nodes that work together.

Epoch: An epoch is one complete pass of the entire training dataset through the model. During each epoch, every training sample is processed, the model calculates the prediction error using a loss function and updates its weights through backpropagation and an optimization algorithm.

- Enable the model to gradually learn complex patterns from the training data.
- Improve model performance by refining weights after each complete pass.
- Help the optimizer converge toward an optimal solution.
- Allow monitoring of training and validation performance after every epoch.

Example: Suppose a training dataset contains 1,000 images and the batch size is 100. The model processes the dataset in 10 batches, updating its weights after each batch. After all 10 batches have been processed, one epoch is completed.

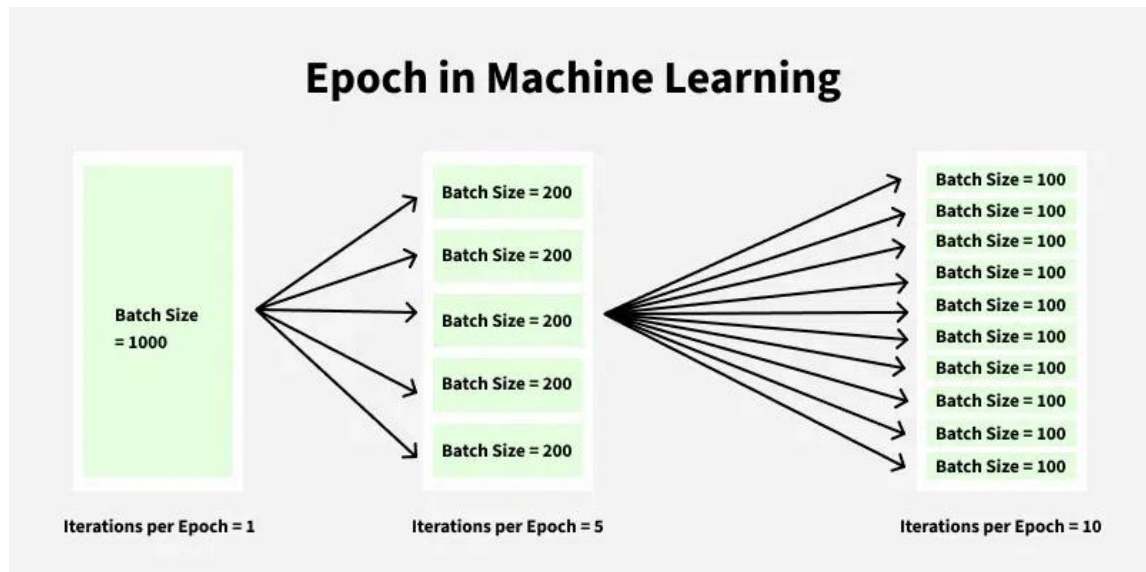


Working: The following steps illustrate how one epoch is completed during deep learning model training.

- **Step 1: Split the Training Dataset:** The complete training dataset is divided into smaller batches based on the selected batch size.
- **Step 2: Process One Batch:** The first batch is passed through the neural network, where the model performs a forward pass to generate predictions.
- **Step 3: Calculate the Loss:** The predicted outputs are compared with the actual labels using a loss function to measure the prediction error.
- **Step 4: Update Model Weights:** Using backpropagation and an optimizer such as SGD or Adam, the model updates its weights to minimize the calculated loss.
- **Step 5: Repeat for All Batches:** The same process is repeated for each remaining batch until every sample in the training dataset has been processed once. After all batches have been processed, one epoch is completed and the next epoch begins if further training is required.

Relationship Between Epochs, Batch Size and Iterations:

- Training a deep learning model involves three closely related concepts: epoch, batch size and iteration. An epoch represents one complete pass through the entire training dataset, while a batch is a subset of the dataset processed at one time. An iteration refers to one weight update after processing a single batch.



As shown in the figure, consider a dataset containing 1,000 training samples:

- When the batch size is 1000, the entire dataset is processed in a single batch, resulting in 1 iteration per epoch.
- When the batch size is 200, the dataset is divided into 5 batches, resulting in 5 iterations per epoch.
- When the batch size is 100, the dataset is divided into 10 batches, resulting in 10 iterations per epoch.
- This shows that reducing the batch size increases the number of iterations required to complete one epoch, while increasing the batch size decreases the number of iterations.
- However, regardless of the batch size, one epoch is completed only after every sample in the training dataset has been processed exactly once.

Relationship: $\text{Iterations per Epoch} = \text{Total Training Samples} \div \text{Batch Size}$

Early Stopping and Learning Rate Scheduling:

Selecting an appropriate number of epochs is essential for training an effective deep learning model.

- Training for too few epochs may result in underfitting, where the model fails to learn meaningful patterns from the training data.

- On the other hand, training for too many epochs can lead to overfitting, causing the model to memorize the training data and perform poorly on unseen data.
- To overcome these challenges, deep learning models commonly use Early Stopping and Learning Rate Scheduling during training.

1. Early Stopping: It is a regularization technique that automatically stops training when the model's performance on the validation dataset stops improving. Instead of training for a fixed number of epochs, it continuously monitors a validation metric, such as validation loss or validation accuracy.

- If the validation metric does not improve for a predefined number of consecutive epochs (called patience), the training process is terminated.
- This helps prevent overfitting while reducing unnecessary computation.
- Reduces training time and computational cost.
- Automatically selects an appropriate number of epochs.
- Improves the model's ability to generalize to unseen data.

2. Learning Rate Scheduling: The learning rate determines how much the model's weights are updated after each iteration. A fixed learning rate may not produce the best results throughout training. Therefore, many deep learning models use Learning Rate Scheduling, which gradually reduces the learning rate as training progresses.

Common learning rate scheduling strategies include:

- Step Decay: Reduces the learning rate after a fixed number of epochs.
- Exponential Decay: Gradually decreases the learning rate after every epoch.
- ReduceLROnPlateau: Reduces the learning rate when the validation performance stops improving.

Loss Functions: Loss functions are a fundamental aspect of machine learning algorithms, serving as the bridge between model predictions and the actual outcomes. They quantify how well or poorly a model is performing by calculating the difference between predicted values and actual values. This difference, or "loss," guides the optimization process to improve

model accuracy. In this article, we will explore various common loss functions used in machine learning, categorized into regression and classification tasks.

Importance of Loss Functions in Machine Learning: Loss functions are integral to the training process of machine learning models. They provide a measure of how well the model's predictions align with the actual data. By minimizing this loss, models learn to make more accurate predictions.

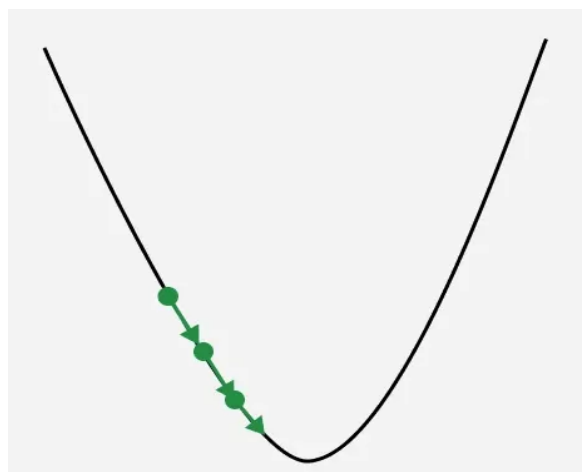
The choice of a loss function can significantly affect the performance of a model, making it crucial to select an appropriate one based on the specific task at hand.

Gradient Descent: Gradient Descent is an iterative optimization algorithm used to minimize a cost function by adjusting model parameters in the direction of the steepest descent of the function's gradient. In simple terms, it finds the optimal values of weights and biases by gradually reducing the error between predicted and actual outputs.

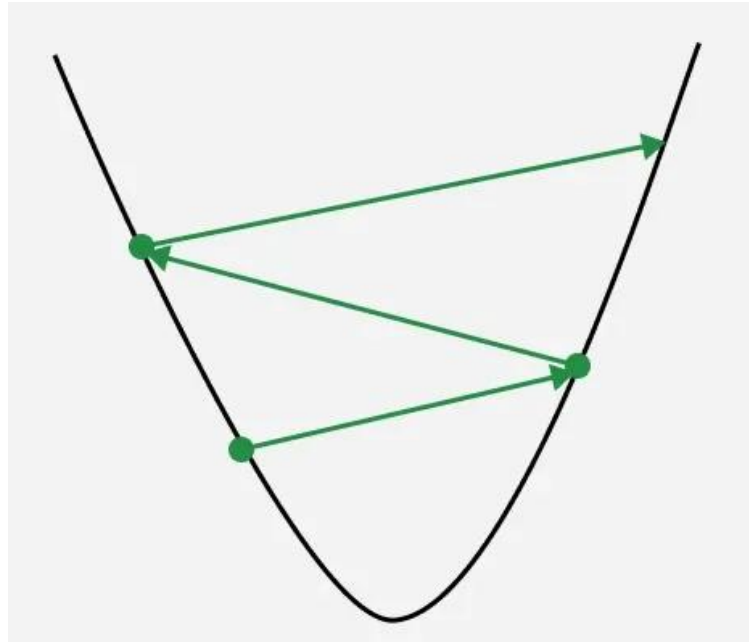
What is Learning Rate?

Learning rate is an important hyperparameter in gradient descent that controls how big or small the steps should be when going downwards in gradient for updating model parameters. It is essential to determine how quickly or slowly the algorithm converges toward minimum of cost function.

1. If Learning rate is too small: The algorithm will take tiny steps during iteration and converge very slowly. This can significantly increase training time and computational cost especially for large datasets.



2. If Learning rate is too big: The algorithm may take huge steps leading overshooting the minimum of cost function without settling. It fail to converge causing the algorithm to oscillate. This process is termed as exploding gradient problem.



In image we can see point got oscillated from right to left with converging to minimum gradient value.

To address these problems, we have some technique that can be used:

- **Weights Regularizations:** The initialization of weights can be adjusted to ensure that they are in an appropriate range. Using a different activation function such as the Rectified Linear Unit (ReLU) can help us to mitigate the vanishing gradient problem.
- **Gradient clipping:** Restrict the gradients to a predefined range to prevent them from becoming excessively large or small.
- **Batch normalization:** It can also help to address these problems by normalizing the input of each layer to prevent activation function from saturating and hence reducing vanishing and exploding gradient problems.

Gradient Descent is an optimization algorithm used to minimize the error of a machine learning model by updating parameters in the direction of decreasing loss.

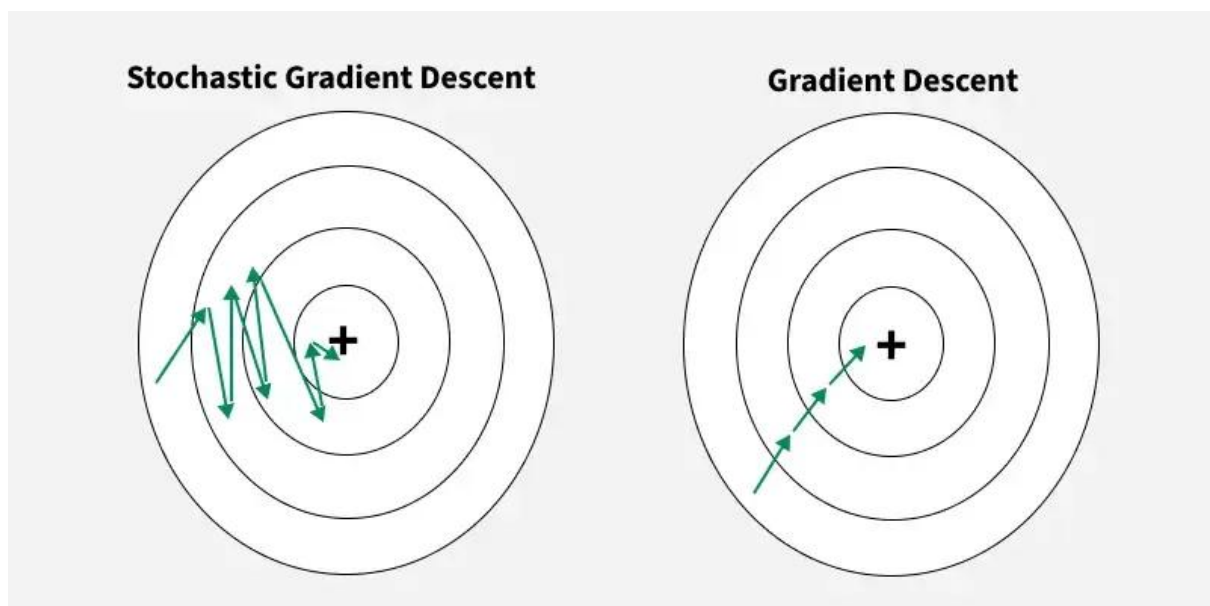
- Minimizes the loss or error of a model
- Updates parameters step by step using gradients
- Helps models like neural networks and regression learn optimal weights
- Variants include Batch Gradient Descent, Stochastic Gradient Descent and Mini Batch Gradient Descent

Parallel & Distributed ML Algorithms - SGD:

Stochastic Gradient Descent is an optimization algorithm used in machine learning, especially for large datasets, that updates model parameters efficiently using small batches or single samples.

- Variant of gradient descent designed for faster and scalable learning
- Updates parameters using one data point or small batches at a time
- Reduces computation compared to full batch gradient descent
- Widely used in deep learning for efficient training

Working of Stochastic Gradient Descent:



Path followed by batch gradient descent vs. path followed by SGD

- In traditional gradient descent, the gradients are computed based on the entire dataset which can be computationally expensive for large datasets.
- In Stochastic Gradient Descent, the gradient is calculated for each training example (or a small subset of training examples) rather than the entire dataset.

Stochastic Gradient Descent update rule is:

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x_i; y_i)$$

Where:

- x_i and y_i represent the features and target of the i-th training example.
- The gradient $\nabla_{\theta} J(\theta; x_i; y_i)$ is now calculated for a single data point or a small batch.

Applications:

- Used in deep learning to train large neural networks efficiently
- Applied in NLP for models like Word2Vec and transformers
- Used in computer vision tasks such as image classification, object detection and segmentation
- Applied in reinforcement learning for models like deep Q-networks and policy gradient methods

Advantages:

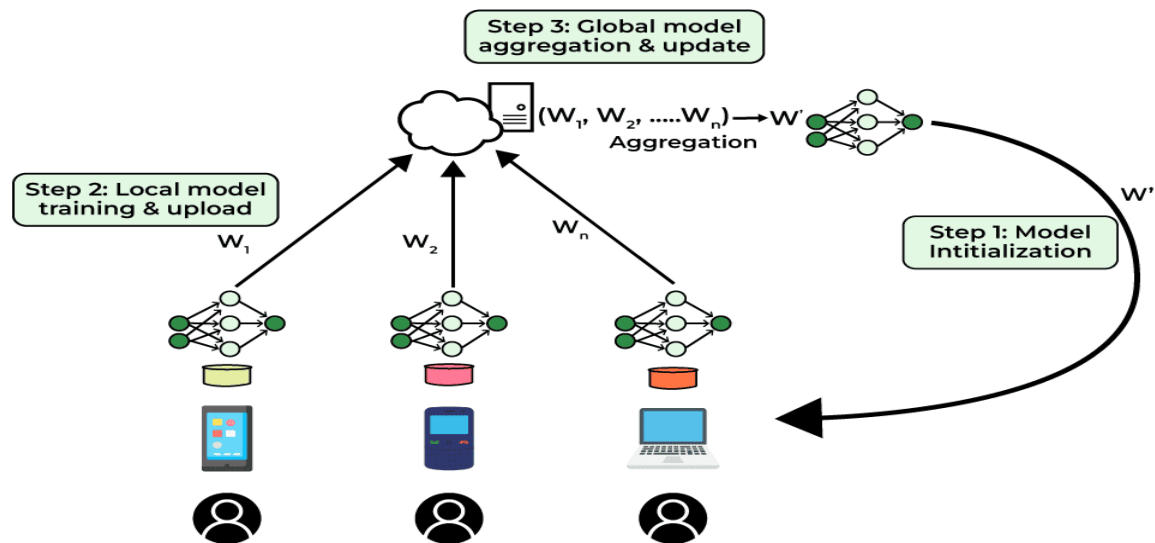
- Faster and more efficient since it updates parameters using one or a few data points
- Requires less memory, making it suitable for large datasets
- Stochastic updates help escape local minima and saddle points
- Supports online learning by updating the model with incoming data

Challenges:

- Updates can be noisy due to using single samples, causing fluctuations in the loss instead of smooth convergence
- Highly sensitive to learning rate; too high may diverge, too low slows down learning
- May take longer to converge overall despite faster individual updates

Parallel & Distributed ML Algorithms - Federated Learning:

Traditional Machine Learning training relied on large datasets, which were stored in centralized locations like data centers, and the goal was to get accurate predictions and generate insights that would profit us in the end. But this approach came with challenges like data storage issues, privacy concerns, and processing. Recently, there has been a key development of the concept of federated learning, which is providing some groundbreaking solutions.



Federated Learning is a technique of training machine learning models on decentralized data, where the data is distributed across multiple devices or nodes, such as smartphones, IoT devices, edge devices, etc. Instead of centralizing the data and training the model in a single location, in Federated Learning, the model is trained locally on each device, and the updates are then aggregated and shared with a central server.

Real-Life Application of Federated Learning: Google Keyboard

- **Data Collection:** Google Keyboard (Gboard) collects data, like the names of restaurants you search for, but keeps it on your phone. Your personal data never leaves your device.
- **Federated Learning in Action:** The model is trained directly on your phone using the data (like your search history). No need to send your data to a central server, keeping it private.

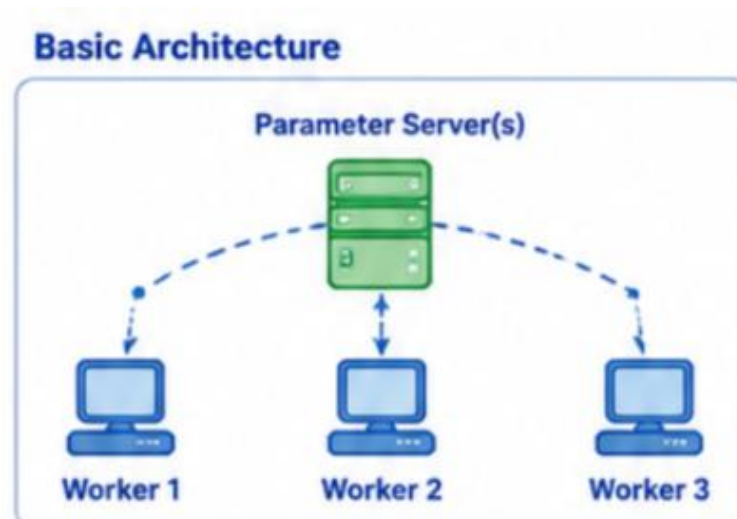
- **Model Updates:** After training locally, the updates to the model are sent to a central server in an encrypted form. The server combines updates from different devices to improve the model.
- **Better Suggestions:** As more updates are gathered, the model gets better at suggesting things, like more accurate restaurant names or typing predictions based on your habits.
- **Privacy and Efficiency:** Your data stays private, and only model updates are shared. This helps save bandwidth and makes everything run more efficiently.

Parameter Servers & All-Reduce Communication

1. Parameter Server:

Definition:

- A Parameter Server is a component used in distributed machine learning to maintain the model variables/parameters while multiple workers perform training.
- Parameter-server training is a data-parallel method used to scale model training across multiple machines. A typical cluster contains workers and parameter servers. The variables are created on the parameter servers and are read and updated by workers during training.



Working:

1. The distributed training cluster contains workers and parameter servers.

2. The model variables are maintained on parameter servers.
3. Workers perform training computations using their assigned data.
4. During training, workers read the model variables from the parameter servers.
5. Workers calculate the required updates during their training steps.
6. The workers update the variables maintained on the parameter servers.
7. The updated variables are then available for subsequent training steps.

TensorFlow describes this architecture explicitly: variables are created on parameter servers and are read and updated by workers in each training step.

Synchronous and Asynchronous Training:

Synchronous training:

Workers coordinate their training so that updates proceed with synchronization between workers.

Asynchronous training:

Workers can independently read and update the variables without synchronizing with each other.

In TensorFlow's `ParameterServerStrategy`, workers read and update variables independently by default, so this style is described as asynchronous training.

Advantages:

- Enables model training across multiple machines.
- Supports multiple worker and parameter-server tasks.
- Provides a mechanism for maintaining model variables in distributed training.
- Can scale distributed training to a large number of workers.

Challenges:

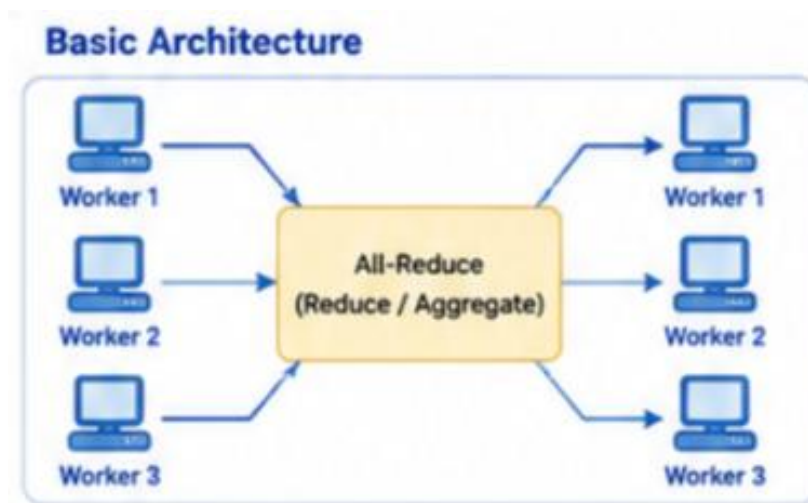
- Communication between workers and parameter servers is required.
- The parameter-server architecture can involve communication and coordination overhead.

- The system must handle failures and unavailable workers/servers in a distributed environment.

2. All-Reduce Communication:

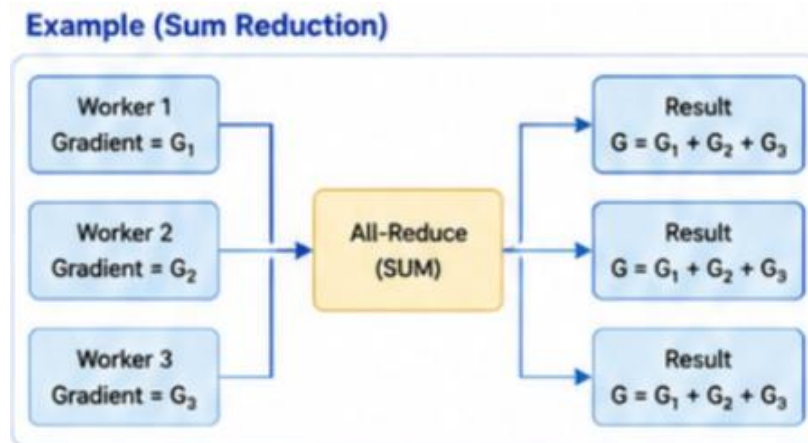
Definition:

All-Reduce is a collective communication operation in distributed training in which values from multiple devices/workers are reduced using an operation such as sum, and the resulting value is made available to all participating devices/workers.



Working:

1. Each worker/device has a local value, such as a gradient.
2. All participating workers take part in the collective communication operation.
3. The local values are reduced/aggregated.
4. A reduction operation such as sum can be performed.
5. The resulting value is made available to all participating workers/devices.
6. The workers can use the common result for distributed training.



TensorFlow's distributed-training documentation describes All-Reduce as aggregating tensors across replicas and making the result available across those replicas.

3. Parameter Server vs All-Reduce:

Parameter Server	All-Reduce
Uses parameter-server nodes	Uses collective communication among workers/devices
Model variables are maintained on parameter servers	Aggregated result is made available to all participants
Workers read and update variables maintained by parameter servers	Workers collectively participate in the reduction
Can support asynchronous training	Commonly used for synchronized distributed training
Central parameter-management architecture	Collective communication approach

TensorFlow's distributed-training documentation describes parameter-server training and All-Reduce as different approaches to distributed training: parameter-server training is associated with asynchronous updates, while synchronous training can use All-Reduce to aggregate results across replicas.